

Authentication & Authorization

What is JWT Creation?

JWT Creation means:

| **The server creates a secure token after the user successfully logs in.**

Imagine you log into your company's website.

```
Username : sayyami  
Password : admin123
```

The server checks the database.

```
Is username correct? ☒ Yes  
Is password correct? ☒ Yes
```

Instead of returning:

```
Login Successful
```

the server returns:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9....
```

This long string is the **JWT Token**.

Why Create a JWT?

Imagine there is no JWT.

Every request would look like:

```
GET /profile
```

```
Username = sayyami  
Password = admin123
```

Next request:

```
GET /orders
```

```
Username = sayyami  
Password = admin123
```

Again:

```
GET /settings
```

```
Username = sayyami  
Password = admin123
```

This is:

- Slow
- Insecure
- Bad design

Instead,

Login once.

```
Username  
Password
```

↓

Server creates JWT.

↓

Client stores JWT.

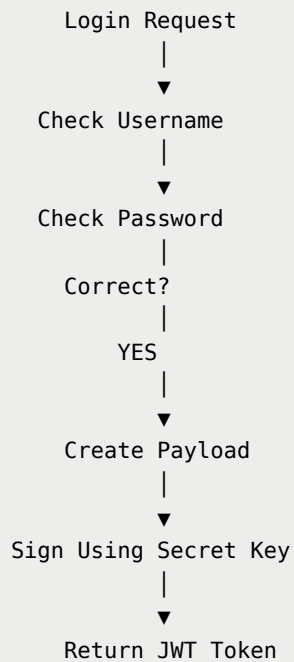
↓

Every future request:

```
Authorization:  
Bearer eyJhbGc...
```

Much better.

JWT Creation Flow



Step 1: Payload

Before creating a JWT, we need some information.

Example:

```
payload= {"id":1,"username":"sayyami","role":"admin"}
```

Think of payload as the **information written on an ID card**.

Example:

```
Employee ID Card

ID : 1
Name : Sayyami
Role : Admin
```

Step 2: Secret Key

The server has a secret.

```
SECRET_KEY="my_super_secret_key"
```

Only the server knows this.

Never send it to React.

Never send it to Android.

Never expose it publicly.

Think of it like the official seal used to sign government documents.

Step 3: Algorithm

```
ALGORITHM="HS256"
```

Think of it as:

```
How should I sign this token?
```

Common algorithms:

```
HS256
HS384
HS512
RS256
```

Most projects use:

```
HS256
```

Step 4: Encode

```
from jose import jwttoken=jwt.encode(payload,SECRET_KEY,algorithm=ALGORITHM)
```

Look carefully.

```
Payload
+
Secret Key
+
Algorithm
↓
JWT
```

Visual Representation

```
{
  id:1
  username:sayyami
  role:admin
}
```

```
}  
  
|  
jwt.encode()  
  
|  
▼  
eyJhbGc...
```

What Actually Happens?

Suppose payload is:

```
{"id":1,"username":"sayyami"}
```

The library creates:

```
Header  
  
+  
  
Payload  
  
+  
  
Signature
```

Result:

```
xxxxx.yyyyy.zzzzz
```

Creation Diagram

```
Payload
  |
  ▼
{
  id:1
  role:admin
}
  |
Secret Key
  |
Algorithm
  |
jwt.encode()
  ▼
JWT Token
```

Now Comes Verification

Someone sends:

```
Authorization:
Bearer eyJhbGc...
```

The server receives it.

Question:

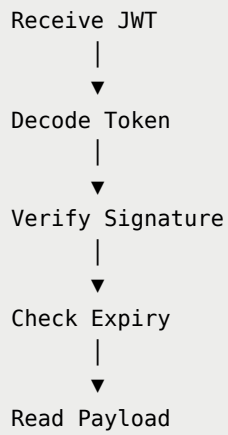
Can we trust it?

Maybe.

Maybe not.

So we verify it.

Verification Flow



Step 1

Receive

Bearer eyJhbGc...

Remove:

Bearer

Now only:

eyJhbGc...

remains.

Step 2

Decode


```
decoded=jwt.decode(token,SECRET_KEY,algorithms=[ALGORITHM])
```

Notice:

We again use

```
SECRET_KEY
```

Why?

Because we need to verify the signature.

Suppose Someone Modifies the Token

Original payload

```
{"role":"user"}
```

Hacker changes it to

```
{"role":"admin"}
```

Will it work?

No.

Because

```
Payload changed
```

```
↓
```

```
Signature changes
```

```
↓
```

Verification fails

Server says:

Invalid Token

Verification Diagram

```
graph TD; A[JWT] --> B[▼]; B --> C[Decode]; C --> D[|]; D --> E[Verify Signature]; E --> F[|]; F --> G[Correct Secret?]; G --> H[|]; H --> I[YES]; I --> J[|]; J --> K[Return Payload]; K --> L[|]; L --> M[NO]; M --> N[|]; N --> O[Reject Token];
```

JWT
▼
Decode
|
Verify Signature
|
Correct Secret?
|
YES
|
Return Payload
|
NO
|
Reject Token

Expiration Verification

Suppose token contains

```
{"exp":1752000000
}
```

Server checks

```
Current Time
>
Expiry Time ?
```

If

```
Expired
```

↓

Reject token.

Complete Verification Logic

```
Receive JWT
↓
Decode
↓
Signature Valid?
↓
YES
↓
Expired?
```

↓
NO
↓
Read User
↓
Allow API

If Any Check Fails
↓
401 Unauthorized

Real FastAPI Code

Create Token

```
from jose import jwt
from datetime import datetime, timedelta

SECRET_KEY="secret123"
ALGORITHM="HS256"
data= {
    "sub": "1", "username": "sayyami", "role": "admin",
    "exp": datetime.utcnow()+timedelta(minutes=30)
}

token=jwt.encode(data, SECRET_KEY, algorithm=ALGORITHM)

print(token)
```

Verify Token

```
decoded=jwt.decode(token,SECRET_KEY,algorithms=[ALGORITHM])  
  
print(decoded)
```

Output

```
{'sub': '1', 'username': 'sayyami', 'role': 'admin', 'exp': 1752050000}  
}
```

Why Can Anyone Decode JWT?

Many beginners think:

JWT = Encryption

Wrong.

JWT is:

Encoding

+

Digital Signature

Anyone can decode:

Header

Payload

But no one can create a **valid signature** without the secret key.

That signature is what protects the token from tampering.

Common JWT Claims

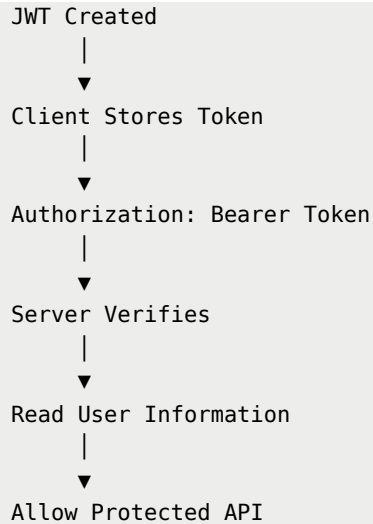
Claim	Meaning
sub	Subject (usually user ID)
exp	Expiration time
iat	Issued at time
nbf	Not before time
iss	Issuer
aud	Audience
jti	Unique token ID

Example:

```
payload= {  
  "sub": "101",  
  "role": "admin",  
  "exp": datetime.utcnow()+  
    timedelta(minutes=30),  
  "iat": datetime.utcnow()  
}
```

Complete Authentication Flow

```
User Login  
  |  
  ▼  
Check Username  
  |  
Check Password  
  |  
  ▼  
Create Payload  
  |  
Add Expiry  
  |  
Sign With Secret Key  
  |  
  ▼
```



Interview Questions

1. What is JWT creation?

JWT creation is the process of generating a signed JSON Web Token after successful user authentication. The server encodes user claims (such as user ID and role) and signs the token using a secret key and a chosen algorithm.

2. What is JWT verification?

JWT verification is the process of checking that:

- The token's signature is valid.
- The token has not expired.
- The token has not been tampered with.

If all checks pass, the server trusts the user information contained in the token.

3. Why do we use a secret key?

The secret key is used to create and verify the token's signature. Without the correct secret key, an attacker cannot generate a valid JWT, even if they know its contents.

4. Can a user read the payload?

Yes. The header and payload are Base64URL-encoded, not encrypted. They can be decoded by anyone, so sensitive information like passwords should never be included.

Final Project Architecture

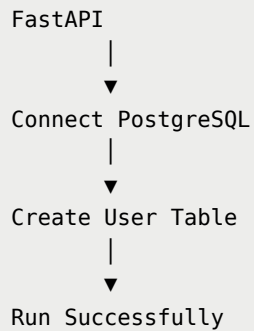
```
fastapi-jwt/
├── app/
│   ├── main.py
│   ├── database.py
│   ├── models.py
│   ├── schemas.py
│   ├── auth.py
│   ├── dependencies.py
│   ├── crud.py
│   ├── config.py
│   └── routers/
│       ├── auth_router.py
│       └── user_router.py
├── requirements.txt
└── .env
```

This is very similar to Spring Boot.

Spring Boot	FastAPI
Controller	Router
Service	CRUD/Service
Repository	SQLAlchemy Session
Entity	Model
DTO	Schema
application.properties	.env

Phase 1 (Today)

Today's goal is only this:



No JWT yet.

Because authentication depends on a working database.

Step 1: Install Packages

Create a virtual environment if you haven't already:

```
python-m venv .venv
```

Activate it.

Windows PowerShell:

```
.venv\Scripts\Activate
```

Now install:

```
pip install fastapi uvicorn sqlalchemy psycopg2-binary python-dotenv passlib[bcrypt] pyth  
on-jose[cryptography]
```

Why these?

Package	Purpose
fastapi	API framework
uvicorn	Server
sqlalchemy	ORM
psycopg2-binary	PostgreSQL driver
python-dotenv	Read <code>.env</code> file
passlib	Password hashing
python-jose	JWT

Step 2: Create PostgreSQL Database

Open pgAdmin.

Create database:

```
fastapi_jwt
```

No tables yet.

Step 3: Create `.env`

```
DATABASE_URL=postgresql://postgres:YOUR_PASSWORD@localhost:5432/fastapi_jwt  
  
SECRET_KEY=ReplaceThisWithARandomLongSecretKey  
  
ALGORITHM=HS256  
  
ACCESS_TOKEN_EXPIRE_MINUTES=30
```

Replace `YOUR_PASSWORD` with your PostgreSQL password.

Why `.env` ?

Never write passwords directly in code.

✗ Bad:

```
DATABASE_URL="postgresql://postgres:1234@localhost/db"
```

✓ Good:

```
DATABASE_URL=os.getenv("DATABASE_URL")
```

Step 4: Create `config.py`

```
from dotenv import load_dotenv
import os
load_dotenv()
DATABASE_URL=os.getenv("DATABASE_URL")
SECRET_KEY=os.getenv("SECRET_KEY")
ALGORITHM=os.getenv("ALGORITHM")
ACCESS_TOKEN_EXPIRE_MINUTES=int(os.getenv("ACCESS_TOKEN_EXPIRE_MINUTES"))
```

What happens?

When Python starts:

```
load_dotenv()
↓
Read .env
↓
Load values into memory
```

Now anywhere in your project:

```
from config import DATABASE_URL
```

Step 5: Create `database.py`

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from sqlalchemy.orm import declarative_base
from config import DATABASE_URL

engine=create_engine(DATABASE_URL)

SessionLocal=sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine)Base=declarative_base()
```

What is `engine` ?

Think of it like this:

```
FastAPI
|
▼
Engine
|
▼
PostgreSQL
```

The engine knows how to communicate with PostgreSQL.

What is `SessionLocal` ?

A session represents one conversation with the database.

```
Open Session
↓
Read
↓
```

Insert

↓

Update

↓

Delete

↓

Close Session

Every API request gets its own session.

Step 6: Create `models.py`

```
from sqlalchemy import Column
from sqlalchemy import Integer
from sqlalchemy import String
from database import Base

class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True, index=True)
    name = Column(String, nullable=False)
    email = Column(String, unique=True, nullable=False)
    password = Column(String, nullable=False)
```

Compare with Spring Boot

Spring:

```
@Entity
public class User {

    @Id
    private Long id;
```

```
    private String name;
}
```

FastAPI:

```
class User(Base):
```

Same concept.

Step 7: Create `main.py`

```
from fastapi import FastAPI
from database import engine
from database import Base

app=FastAPI()
Base.metadata.create_all(bind=engine)

@app.get("/")
def home():
    return {"message":"FastAPI JWT Course"}
```

What does `create_all()` do?

```
Looks at Models
↓
User Model
↓
Create users table
```

Exactly once.

If the table already exists:

```
Nothing happens.
```

Step 8: Run

```
python-m uvicorn app.main:app--reload
```

Check PostgreSQL

Refresh pgAdmin.

You should now see:

```
fastapi_jwt
↓
Schemas
↓
Tables
↓
users
```

🎉 Congratulations! Your FastAPI application is now connected to PostgreSQL, and SQLAlchemy has created the `users` table.

Why We're Doing It This Way

Many tutorials jump straight into JWT.

Professional development follows this order:

```
Database
↓
Models
↓
Schemas
↓
```

Password Hashing

↓

Register

↓

Login

↓

JWT

↓

Protected Routes

If the database isn't working, authentication cannot work.

Part 2: User Registration

Goal:

POST /register

↓

Hash Password

↓

Save User

↓

PostgreSQL

Why Do We Need Schemas?

FastAPI equivalent:


```
from pydantic import BaseModel
class UserCreate(BaseModel):
    name:str
    email:str
    password:str
```

This is called a **Schema**.

Create `schemas.py`

```
from pydantic import BaseModel
from pydantic import EmailStr

class UserCreate(BaseModel):
    name:str
    email:EmailStr
    password:str
```

Why EmailStr?

Normal string:

```
email:str
```

Accepts:

```
hello
abc
xyz
```

Bad.

Using:

```
EmailStr
```

Accepts:

```
sayyami@gmail.com
```

Rejects:

```
abc  
hello  
123
```

Automatic validation.

Step 2: Password Hashing

Never store:

```
admin123
```

Store:

```
$2b$12$kd fkjshdf...
```

Create:

```
security.py
```

```
from passlib.context import CryptContext  
  
pwd_context=CryptContext(  
    schemes=["bcrypt"],  
    deprecated="auto"  
)
```

```
def hash_password(password:str):  
    return pwd_context.hash(password)
```

Test It

```
from security import hash_password  
  
print(hash_password("admin123")  
)
```

Output:

```
$2b$12$8Jdkfhjsjdf...
```

Every run produces a different hash.

That's normal.

Step 3: Create Database Dependency

Create:

```
dependencies.py
```

```
from database import SessionLocal  
def get_db():  
    db=SessionLocal()  
    try:  
        yield db  
  
    finally:  
        db.close()
```

Why?

Without this:

```
Open DB
Open DB
Open DB
Open DB
```

Connections never close.

Bad.

With dependency:

```
Open Session
↓
Use Session
↓
Close Session
```

Step 4: Create CRUD Logic

Create:

```
crud.py
```

Create User Function

```
from models import User

def create_user(
    db,
```

```
name,  
email,  
password  
):  
    user=User(  
        name=name,  
        email=email,  
        password=password  
    )  
  
    db.add(user)  
    db.commit()  
    db.refresh(user)  
    return user
```

What is happening?

Create object

```
user=User(...)
```

Memory only.

Not saved yet.

Add

```
db.add(user)
```

SQLAlchemy starts tracking it.

Still not saved.

Commit

```
db.commit()
```

Equivalent SQL:

```
INSERT INTO users(...)VALUES(...)
```

Now saved.

Refresh

```
db.refresh(user)
```

Reload from database.

Gets generated ID.

Step 5: Create Register Route

Create:

```
routers/auth_router.py
```

```
from fastapi import APIRouter
from fastapi import Depends

from sqlalchemy.orm import Session
from schemas import UserCreate
from security import hash_password
from crud import create_user
from dependencies import get_db

router=APIRouter()
```

Register API

Hash Password

```
hashed_password=hash_password(user.password
)
```

Save User

```
new_user=create_user(db=db,name=user.name,email=user.email,password=hashed_password
)
```

Return Response

```
return {"message":"User Registered","id":new_user.id
}
```

Full Register Endpoint

```
@router.post("/register")defregister(user:UserCreate,db:Session=Depends(get_db)
):hashed_password=hash_password(user.password
    )new_user=create_user(db=db,name=user.name,email=user.email,password=hashed_password
    )return {"message":"User Registered","id":new_user.id
    }
```

Add Router to Main

In `main.py`

```
fromrouters.auth_routerimportrouterasauth_router
```

Add:

```
app.include_router(auth_router)
```

Test

Swagger:

```
http://127.0.0.1:8000/docs
```

Call:

```
POST /register
```

Request:

```
{
  "name": "Sayyami",
  "email": "sayyami@gmail.com",
  "password": "admin123"
}
```

Response:

```
{
  "message": "User Registered",
  "id": 1
}
```

Check PostgreSQL

Run:


```
select*from users;
```

Example:

```
id | name      | email
---+-----+-----
1  | Sayyami   | sayyami@gmail.com
```

Password column:

```
$2b$12$asd98asd...
```

Not:

```
admin123
```

Perfect.

What You Learned

Pydantic Schema

Equivalent to DTO.

```
UserCreate
```

bcrypt

Converts

```
admin123
```

into

```
$2b$12$...
```

Depends()

Equivalent to dependency injection.

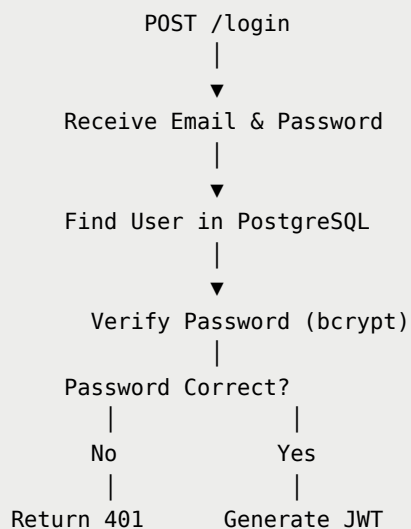
```
Depends(get_db)
```

SQLAlchemy Session

```
add()  
commit()  
refresh()
```

Until now, users can **register**, but they cannot **log in**. In this lesson, we'll build the login flow and generate a JWT.

What We'll Build



Return Access Token

This is exactly how authentication works in most real-world applications.

Before Coding, Understand the Login Process

Imagine your database contains:

ID	Name	Email	Password (Hashed)
1	Sayyami	sayyami@gmail.com	\$2b\$12\$...

The user sends:

```
{
  "email": "sayyami@gmail.com",
  "password": "admin123"
}
```

The server **does not compare**:

```
admin123 == $2b$12$...
```

because that will always be false.

Instead:

```
User Password
  |
  ▼
bcrypt.verify()
  |
  ▼
Compare with Stored Hash
  |
```

▼
True or False

This is why we hash passwords during registration.

Step 1: Create Login Schema

Open `schemas.py`

Add:

```
class UserLogin(BaseModel):  
    email:EmailStr  
    password:str
```

This schema represents the request body for `/login`.

Step 2: Create Token Schema

Also add:

```
class Token(BaseModel):  
    access_token:str  
    token_type:str
```

This represents the response returned after login.

Example:

```
{  
  "access_token": "eyJhbGc....",  
  "token_type": "bearer"  
}
```

Step 3: Verify Password

Open `security.py`

Currently you have:

```
def hash_password(password):
```

Now add:

```
def verify_password(plain_password,hashed_password):  
    return pwd_context.verify(plain_password,hashed_password)
```

What happens?

User enters:

```
admin123
```

Database stores:

```
$2b$12$ABCDEF.....
```

Calling:

```
verify_password("admin123","$2b$12$ABCDEF....."  
)
```

returns:

```
True
```

Wrong password:

```
False
```

Step 4: Find User by Email

Open `crud.py`

Add:

```
from models import User
```

Now create:

```
def get_user_by_email(
    db,
    email
):
    return (
        db.query(User).filter(User.email==email).first()
    )
```

What SQLAlchemy does

This code:

```
db.query(User)
.filter(User.email==email)
.first()
```

is equivalent to SQL:

```
SELECT*FROM users WHERE email='sayyami@gmail.com'LIMIT1;
```

If found:

Returns User object

Otherwise:

Returns `None`

Step 5: JWT Creation

Now comes the exciting part.

Create `auth.py`

Imports:

```
from datetime import datetime
from datetime import timedelta
from jose import jwt

from config import (SECRET_KEY,
                    ALGORITHM,
                    ACCESS_TOKEN_EXPIRE_MINUTES
)
```

Create Function

```
def create_access_token(data:dict):
```

Notice:

We're sending a dictionary.

Example:

```
{"sub":"1","email":"sayyami@gmail.com"}
}
```

Copy Payload

```
to_encode=data.copy()
```

Why?

Because we don't want to modify the original dictionary.

Add Expiration

```
expire= (  
    datetime.utcnow()+timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES  
    )  
)
```

Suppose:

Current time

10:00

Expiry

10:30

Add Expiration to Payload

```
to_encode.update(  
    {"exp":expire}  
)
```

Payload becomes:

```
{"sub":"1","email":"sayyami@gmail.com","exp": ...  
}
```

Encode

```
token=jwt.encode(to_encode,SECRET_KEY,algorithm=ALGORITHM  
)
```


Return:

```
return token
```

Final Function

```
def create_access_token(data:dict):
    to_encode=data.copy()
    expire= (datetime.utcnow()+
    timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    )
    to_encode.update({"exp":expire})
    return jwt.encode(to_encode,SECRET_KEY,algorithm=ALGORITHM)
```

Step 6: Login API

Open

```
routers/auth_router.py
```

Imports:

```
from fastapi import HTTPException
from schemas import UserLogin
from security import verify_password
from crud import get_user_by_email
from auth import create_access_token
```

Login Endpoint

```
@router.post("/login")
def login(user:UserLogin,db:Session=Depends(get_db)
):
```

Find User

```
db_user=get_user_by_email(db,user.email)
```

If user doesn't exist

```
if not db_user:
    raiseHTTPException(status_code=401,detail="Invalid Credentials"
    )
```

Verify Password

```
if not verify_password(user.password,db_user.password):
    raiseHTTPException(status_code=401,detail="Invalid Credentials"
    )
```

Generate JWT

```
token=create_access_token(
    {"sub":str(db_user.id),"email":db_user.email
    }
)
```

Notice

We store:

```
sub
```

This is a standard JWT claim representing the subject (usually the user ID).

Return Token

```
return {"access_token":token,"token_type":"bearer"}
}
```

Final Login API

```
@router.post("/login")
def login(user: UserLogin, db: Session = Depends(get_db)):
    db_user = get_user_by_email(db, user.email)

    if db_user is None:
        raise HTTPException(status_code=400, detail="Invalid credentials")

    try:
        ph.verify(db_user.password, user.password)
    except VerifyMismatchError:
        raise HTTPException(status_code=400, detail="Invalid credentials")

    token = create_access_token(
        {
            "sub": str(db_user.id),
            "email": db_user.email
        }
    )

    return {
        "access_token": token,
        "token_type": "bearer"
    }
```

Test in Swagger

Register

```
{
    "name": "Sayyami",
    "email": "sayyami@gmail.com",
}
```

```
"password": "admin123"
}
```

Login

```
{
  "email": "sayyami@gmail.com",
  "password": "admin123"
}
```

Response:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs... ",
  "token_type": "bearer"
}
```

🎉 Your first real JWT has been created!

Where Is This Token Stored?

The backend **does not store the JWT**.

The client (React, Postman, mobile app) stores it.

Example in React:

```
Login
↓
Receive JWT
↓
Store in memory, cookie, or local storage (depending on your security strategy)
↓
```

Send with every protected request

Each protected request includes: